Avaliação — Navegação Contínua do Pioneer 3DX na Linha de Montagem

Atividade avaliativa em equipe · Linguagem: Python · Valor: 10,0 pontos (+ bônus de até 1,0)

1. Contexto e objetivos

Nesta avaliação, o robô Pioneer 3DX deve controlar as velocidades linear e angular **ao mesmo tempo** e percorrer a linha de montagem **entre os quatro blocos, por dentro e por fora**, em **linhas retas**. Nas quinas, ele faz **curvas suaves**, sem ondulações, e em cada estação para com a orientação de atendimento. Ao final, a equipe compara essa estratégia com a lei polar e com a de duas fases.

Ao final da atividade, a equipe deverá ser capaz de:

1. Implementar uma lei de controle de pose em coordenadas polares para um robô com restrição não holonômica.
2. Implementar um seguidor de caminho (retas e arcos) que combina antecipação da curvatura com realimentação dos erros lateral e de rumo.
3. Ajustar ganhos respeitando as condições de estabilidade e os limites de velocidade do P3DX.
4. Avaliar a robustez do controle a ruído de odometria e de atuação.
5. Comparar estratégias de navegação usando métricas quantitativas e defender uma escolha com dados.

2. Enunciado

A atividade tem quatro tarefas, todas no mesmo programa e com a mesma estrutura de laço do `Ex02_Controlador`.

Tarefa A — Controle contínuo até uma pose. Implemente a lei de controle em coordenadas polares, em que o robô usa u e w ao mesmo tempo para chegar ao ponto (xd, yd) com orientação final ψf:

$$\rho = \sqrt{\Delta x^2 + \Delta y^2}, \quad \alpha = \operatorname{atan2}(\Delta y, \Delta x) - \psi, \quad \beta = \psi_f - \psi - \alpha$$

$$u = k_\rho \rho, \qquad w = k_\alpha \alpha + k_\beta \beta, \qquad k_\rho > 0, \;\; k_\beta < 0, \;\; k_\alpha - k_\rho > 0$$

Teste a lei com três situações partindo de (0, 0, 0°): destino à frente (3, 1, 0°), destino ao lado (0, 2, 90°) e destino atrás (−2, 0, 180°). Explique o que acontece quando |α| > 90° e como a equipe tratou esse caso.

Tarefa B — Roteiro entre os blocos, por dentro e por fora. Percorra o roteiro da Seção 3 sobre o **caminho de referência**: retas entre os pontos e, em cada quina (C1 a C8), um arco de raio 0,5 m tangente às duas retas. O robô deve usar u e w ao mesmo tempo, fazer as quinas em curva **sem parar** e **sem ondular** em torno das retas (desvio máximo do caminho ≤ 5 cm). Nas estações, ele para com a orientação de atendimento. Uma lei adequada é a do seguidor de caminho, em que e_y é o erro lateral, e_ψ o erro de rumo em relação à tangente do caminho e κ a curvatura (0 nas retas e ±1/R nos arcos):

$$u = u_{ref} \max(\cos e_\psi,\ 0), \qquad w = u\,\kappa + k_\psi\, e_\psi - k_y\, u\, e_y$$

Mostre também o que acontece se a lei polar da Tarefa A for usada diretamente no mesmo roteiro.

Tarefa C — Robustez a ruído. Repita o roteiro com ruído na odometria e nos atuadores (valores na Seção 4). Faça 10 execuções com sementes aleatórias diferentes e informe média e desvio padrão de cada métrica.

Tarefa D — Comparação de estratégias. Execute o mesmo roteiro, com e sem ruído, com três estratégias: o seguidor de caminho da Tarefa B, a lei polar da Tarefa A (com as quinas como pontos de passagem) e o controlador em duas fases do desafio anterior. Compare as três pelas métricas da Seção 5 e indique qual a equipe usaria numa linha de montagem real, justificando com os dados.

3. Dados de entrada

O robô parte da **Base**, em (0, −5), com $\psi = 90°$, e **volta para a Base** no fim do roteiro, parando de novo voltado para 90°: a chegada é a própria Base, na mesma pose da partida. O roteiro passa **entre os quatro blocos, por dentro e por fora**: cada bloco é atendido por uma estação **interna**, no corredor central (x = ±1,4 m, a 0,6 m da face), e por uma **externa**, na borda do cenário (x = ±5 m, a 1 m da face), as duas voltadas para o bloco.

O robô vai pela borda de baixo até C1 e contorna a coluna da direita: sobe por fora (SEe e NEe), passa por cima dos blocos (C2 e C3) e desce por dentro (NEi e SEi). Pelo corredor de baixo (C4 e C5), passa para a coluna da esquerda, sobe por dentro (SOi e NOi), passa por cima (C6 e C7) e desce por fora (NOe e SOe). Por fim, contorna a quina C8 e volta pela borda de baixo até a Base. São 17 trechos: 58,8 m de polilinha, ou 57,1 m com as curvas. O cenário e os obstáculos são os do arquivo `cenario_linha_montagem.py`.

#	Destino	x (m)	y (m)	ψ final	Tipo
1	C1 (quina)	5	−5	—	Curva, sem parar
2	Estação SEe (fora)	5	−2,5	180°	Parada
3	Estação NEe (fora)	5	2,5	180°	Parada
4	C2 (quina)	5	5	—	Curva, sem parar
5	C3 (quina)	1,4	5	—	Curva, sem parar
6	Estação NEi (dentro)	1,4	2,5	0°	Parada
7	Estação SEi (dentro)	1,4	−2,5	0°	Parada
8	C4 (quina)	1,4	−4,4	—	Curva, sem parar
9	C5 (quina)	−1,4	−4,4	—	Curva, sem parar
10	Estação SOi (dentro)	−1,4	−2,5	180°	Parada
11	Estação NOi (dentro)	−1,4	2,5	180°	Parada
12	C6 (quina)	−1,4	5	—	Curva, sem parar
13	C7 (quina)	−5	5	—	Curva, sem parar
14	Estação NOe (fora)	−5	2,5	0°	Parada
15	Estação SOe (fora)	−5	−2,5	0°	Parada
16	C8 (quina)	−5	−5	—	Curva, sem parar
17	Base (chegada)	0	−5	90°	Parada final

Parâmetros do Pioneer 3DX: velocidade linear máxima de 0,75 m/s, velocidade angular máxima de 100°/s e controle aplicado ao centro do eixo das rodas.

4. Requisitos e restrições

Item	Valor exigido
Linguagem e base	Python, com a estrutura de laço do `Ex02_Controlador` (rGetSensorData → controlador → pSC.Ud → rSendControlSignals → mCADplot)
Frequência de controle	10 Hz
Saturação	$\|u\| \le 0{,}75$ m/s e $\|w\| \le 100°$/s, aplicada no controlador, preservando a razão u/w
Chegada nas estações	$\rho < 3$ cm e $\|\psi_f - \psi\| < 2°$, com o robô parado
Caminho de referência	retas entre os pontos; em cada quina, arco de raio 0,5 m tangente às duas retas
Quinas (C1 a C8)	o robô faz a curva sem parar e sem girar no lugar
Desvio do caminho (Tarefa B)	no máximo 5 cm, sem ondulações em torno das retas
Ruído de odometria (Tarefa C)	gaussiano, $\sigma = 1$ cm em x e y e $\sigma = 0{,}5°$ em ψ, a cada leitura
Ruído de atuação (Tarefa C)	multiplicativo: u e w multiplicados por $(1 + N(0; 0{,}05))$
Segurança	nenhum trecho pode tocar um obstáculo; a margem mínima de 0,35 m é verificada pelo programa
Tempo máximo	300 s para o roteiro completo

Na Tarefa C, o controlador só enxerga a pose com ruído, mas as métricas são calculadas com a pose verdadeira do simulador. Para isso, o programa deve guardar as duas poses.

5. Métricas de desempenho

O programa deve calcular as seis métricas abaixo e imprimi-las numa tabela no terminal ao final de cada execução.

Métrica	Definição	Unidade
Tempo total	do início até a parada final, de volta à Base	s
Comprimento percorrido	soma de $\sqrt{(\Delta x^2 + \Delta y^2)}$ entre amostras	m
Erro de pose nas estações	ρ e $\|\psi_f - \psi\|$ na parada de cada estação	cm, °
Desvio da referência	maior distância entre o robô e o caminho de referência, retas e arcos (máx.), e sua média quadrática (RMS)	cm
Folga mínima	menor distância entre o robô e um obstáculo	m
Esforço de controle	integrais de u^2 e de w^2 no tempo, reportadas separadamente	m^2/s, rad^2/s

Na Tarefa C, cada métrica é apresentada como média ± desvio padrão das 10 execuções.

6. Entregáveis

A entrega é feita por equipe, em um único arquivo .zip, até a data definida pelo professor.

1. **Código-fonte** comentado em Python, que rode com `python <arquivo>.py` usando apenas numpy e matplotlib. Uma variável no topo deve selecionar a tarefa (A, B, C ou D) e ligar ou desligar o ruído.
2. **Figuras**: a trajetória de cada tarefa com o caminho de referência, os gráficos de erros e sinais de controle, e um gráfico comparando as três estratégias.
3. **Relatório** de no máximo 6 páginas, com a lei de controle e os ganhos escolhidos, a tabela de métricas e as respostas às questões orientadoras.

4. **Vídeo** de até 4 minutos com a simulação do roteiro completo da Tarefa B.

7. Critérios de avaliação

A nota vai de 0 a 10. Uma execução em que o robô toca um obstáculo zera a tarefa correspondente.

Critério	Pontos	Para nota máxima
Tarefa A — lei contínua	2,0	Os três casos convergem dentro da tolerância; os ganhos respeitam as condições de estabilidade; o caso com $\|\alpha\| > 90°$ é tratado e explicado
Tarefa B — roteiro	2,5	Os 17 trechos são concluídos em até 300 s; o robô para nas 8 estações e na volta à Base dentro da tolerância; faz as quinas em curva, sem parar; desvio máximo do caminho ≤ 5 cm, sem ondulações
Tarefa C — robustez	2,0	10 execuções com ruído, pose verdadeira separada da estimada, métricas como média ± desvio padrão
Tarefa D — comparação	1,5	Mesmas condições nas três estratégias; tabela e gráfico comparativos; escolha justificada com os números
Análise e relatório	1,5	Respostas às questões orientadoras sustentadas pelos resultados da própria equipe
Código e vídeo	0,5	Código legível e comentado, que roda sem ajustes; vídeo mostra o roteiro completo
Total	**10,0**	

8. Questões orientadoras para a análise

1. Como os ganhos k_ψ e k_y do seguidor de caminho afetam o desvio em relação às retas? O que acontece sem a correção lateral ($k_y = 0$) e com k_y muito alto?
2. A saturação de u e w altera a garantia de estabilidade? Por que, no seguidor de caminho, convém limitar u antes de calcular w?
3. Por que a lei polar não é definida em $\rho = 0$? Como a equipe evitou oscilações perto das estações?
4. Qual é o efeito do raio das curvas nas quinas sobre o tempo total, o desvio do caminho e a velocidade angular? Que limite físico do P3DX restringe o raio mínimo?
5. Com ruído, qual estratégia teve o maior aumento no erro nas estações? Por quê?
6. Por que a lei polar, que funciona bem em espaço livre (Tarefa A), faz ondulações no roteiro? Em quais faixas (internas ou externas) isso é mais perigoso, e por quê? Em que situações de uma linha de montagem real cada estratégia é vantagem ou risco?

Referências

- SIEGWART, R.; NOURBAKHSH, I. R.; SCARAMUZZA, D. Introduction to Autonomous Mobile Robots. 2. ed. MIT Press, 2011. Cap. 3: controle cinemático em coordenadas polares.
- AICARDI, M. et al. Closed loop steering of unicycle-like vehicles via Lyapunov techniques. IEEE Robotics & Automation Magazine, v. 2, n. 1, 1995.
- Código-base da disciplina: `Ex02_Controlador.py`, `pioneer3dx.py` e `cenario_linha_montagem.py`.